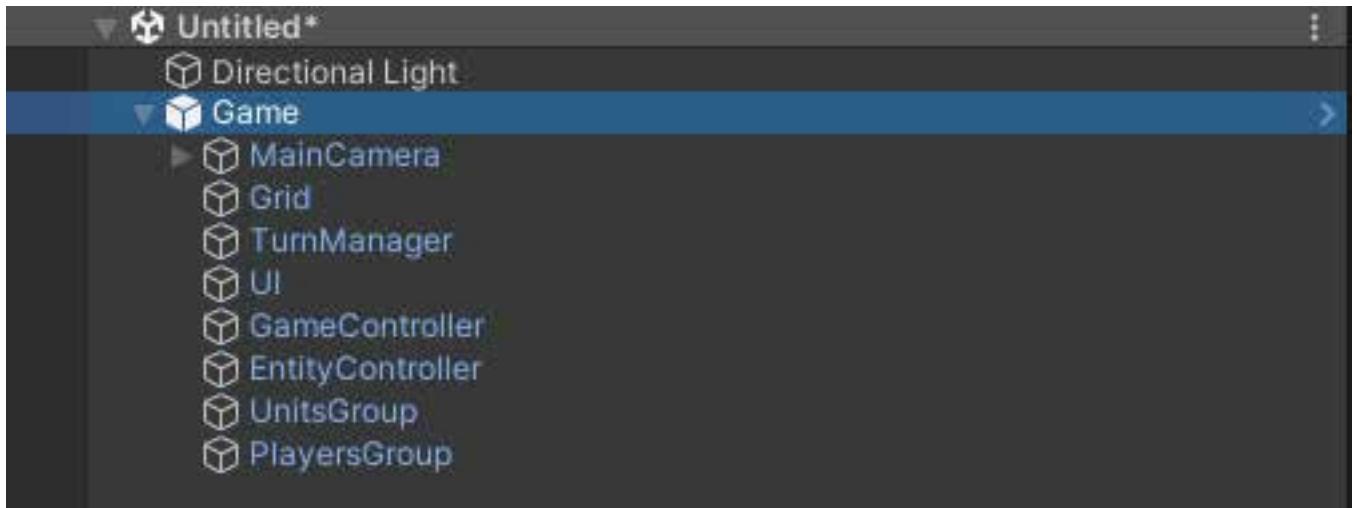


Getting Started

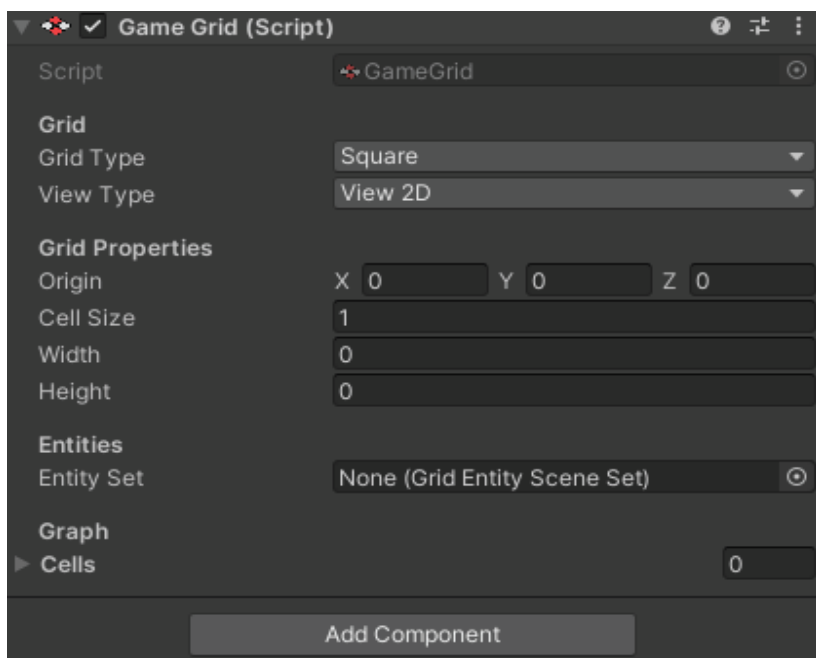
To get started we can create a new scene and drag the Game prefab into the scene hierarchy.



The Game prefab contains a couple of necessary objects, as well as a couple that are handy to create our first game.

The Game prefab already contains a camera and event system. So if you have one from the new scene, you can delete it.

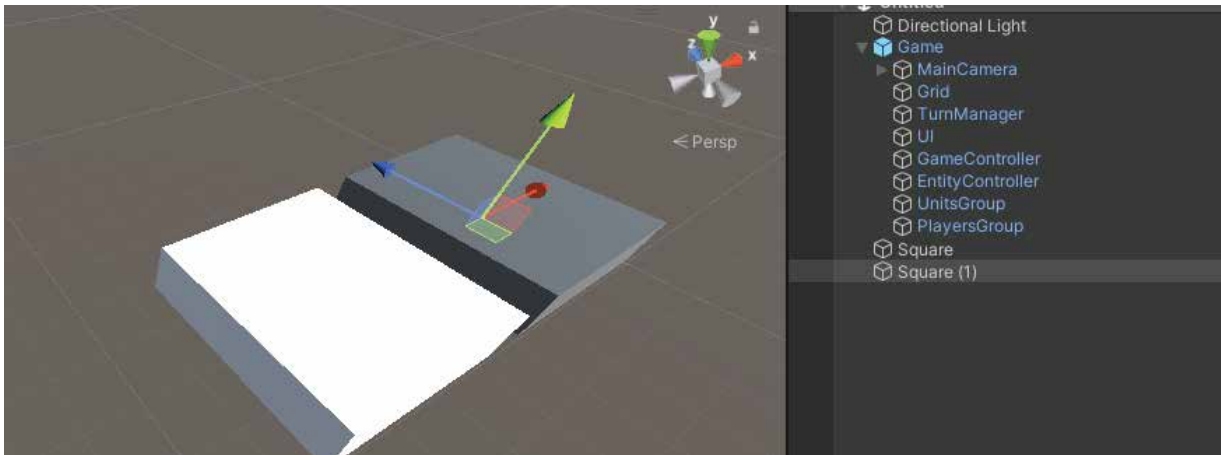
Inside the Game prefab there is a child called Grid. This object has the GameGrid component which is responsible for keeping track of all the cells in our grid, as well as build the graph that is required for pathfinding.



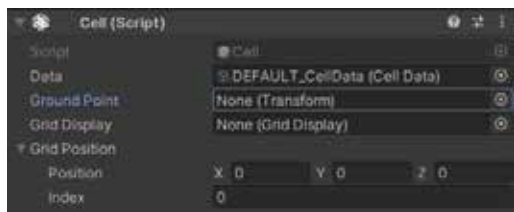
At the moment we have not configured this at all so it will not have any cells

Creating a Cell for the GameGrid

To create a Cell we can simply create a new gameobject in the scene



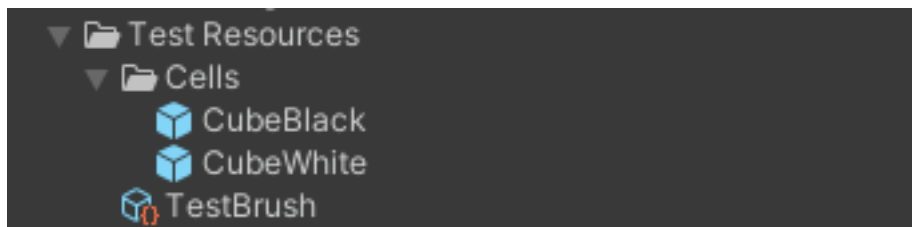
Here I created two 3D primitives and adjusted the y-scale a little bit to make them look flat. I also duplicated the cube and added a different material so that we have a variation. Add a Cell component to the gameobjects.



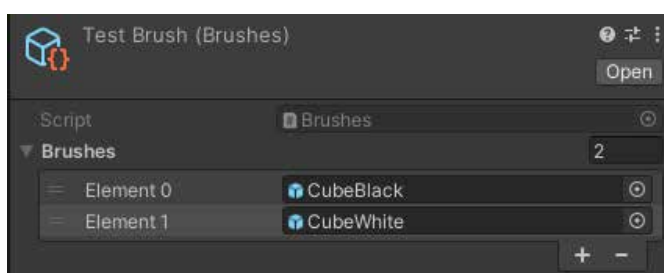
Each Cell object requires the Cell Component to work in the grid. It is required that we have a CellData asset on the Cell, so let's just add the default one for now. We will come back to this. The other fields can be left "None" for now.

Drag these two objects to the Project hierarchy to make them into prefabs, and delete them from the scene.

Now that we have something to build with it's time to start looking at the Grid Builder tool. Right click in the project hierarchy and go to Create ->Gridr/Brushes. This will created a Brushes Scriptable Object that will contain the prefabs we want to use for our grid.



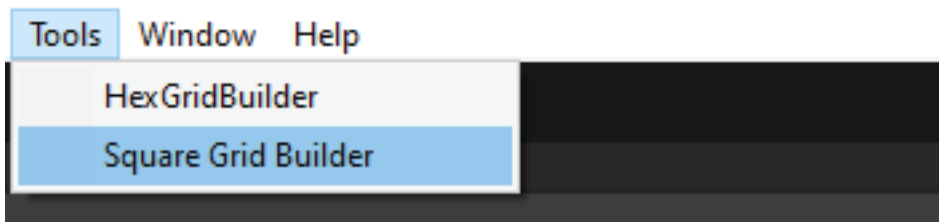
This is what our project should look like right now.



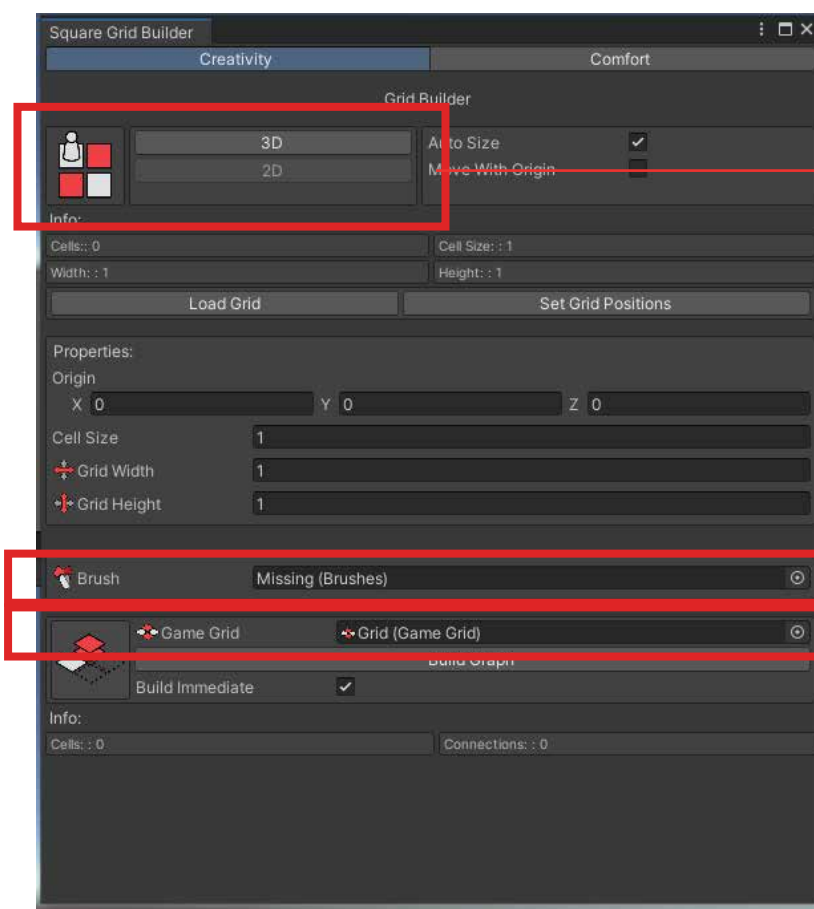
Select the newly created Brushes asset and add the prefabs to the list of game objects.

Using the Grid Builder Tool

To get started with the Grid Builder tool, select tools -> Square Grid Builder



This window will appear



Since we are working in 3D, select the 3D option

In the Brushes field, select the brush we created

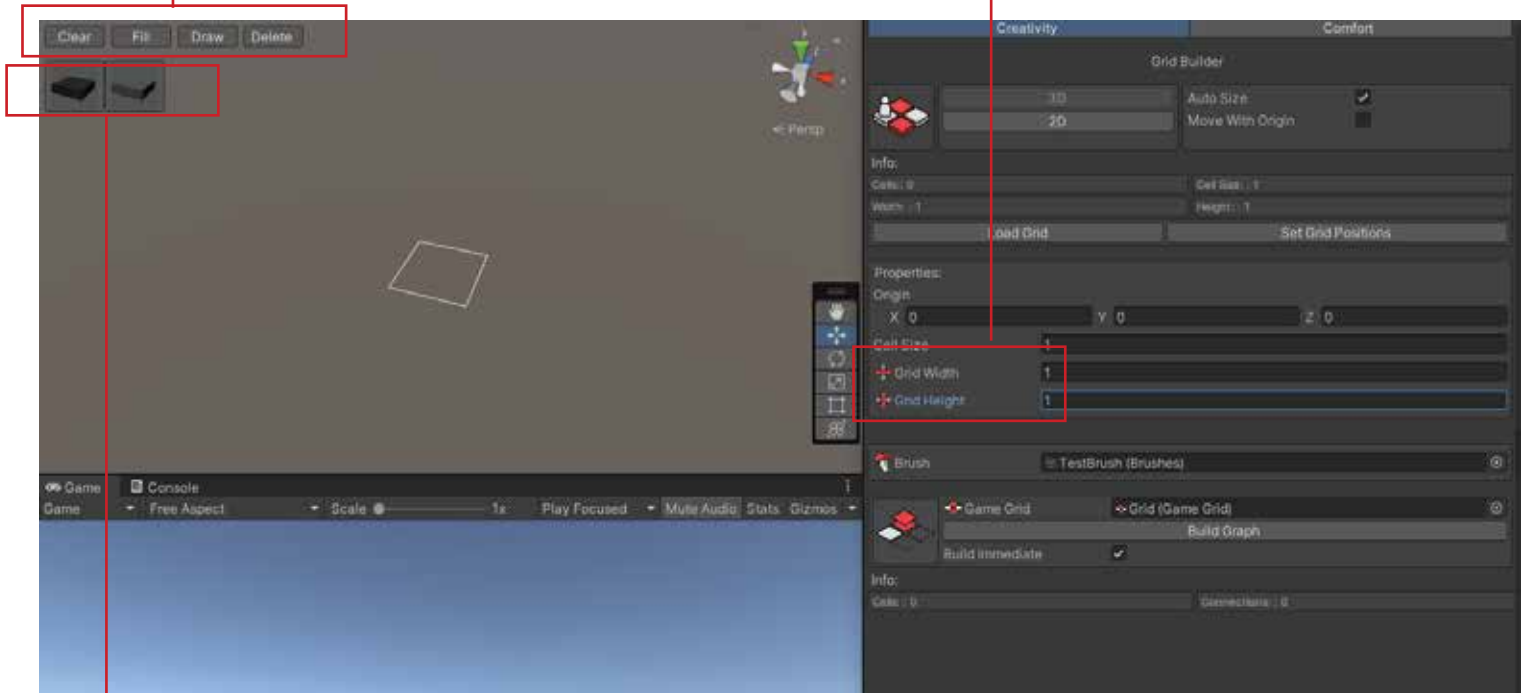
The GameGrid should be loaded automatically, but if it's not make sure to add it here. It is required we have GameGrid in the scene to create a grid with the tool.

Using the Grid Builder Tool

Right now our scene should look something like this

These are the control buttons. Draw is selected by default.

The width and height of the grid in units of Cells. Lets set these to 10.

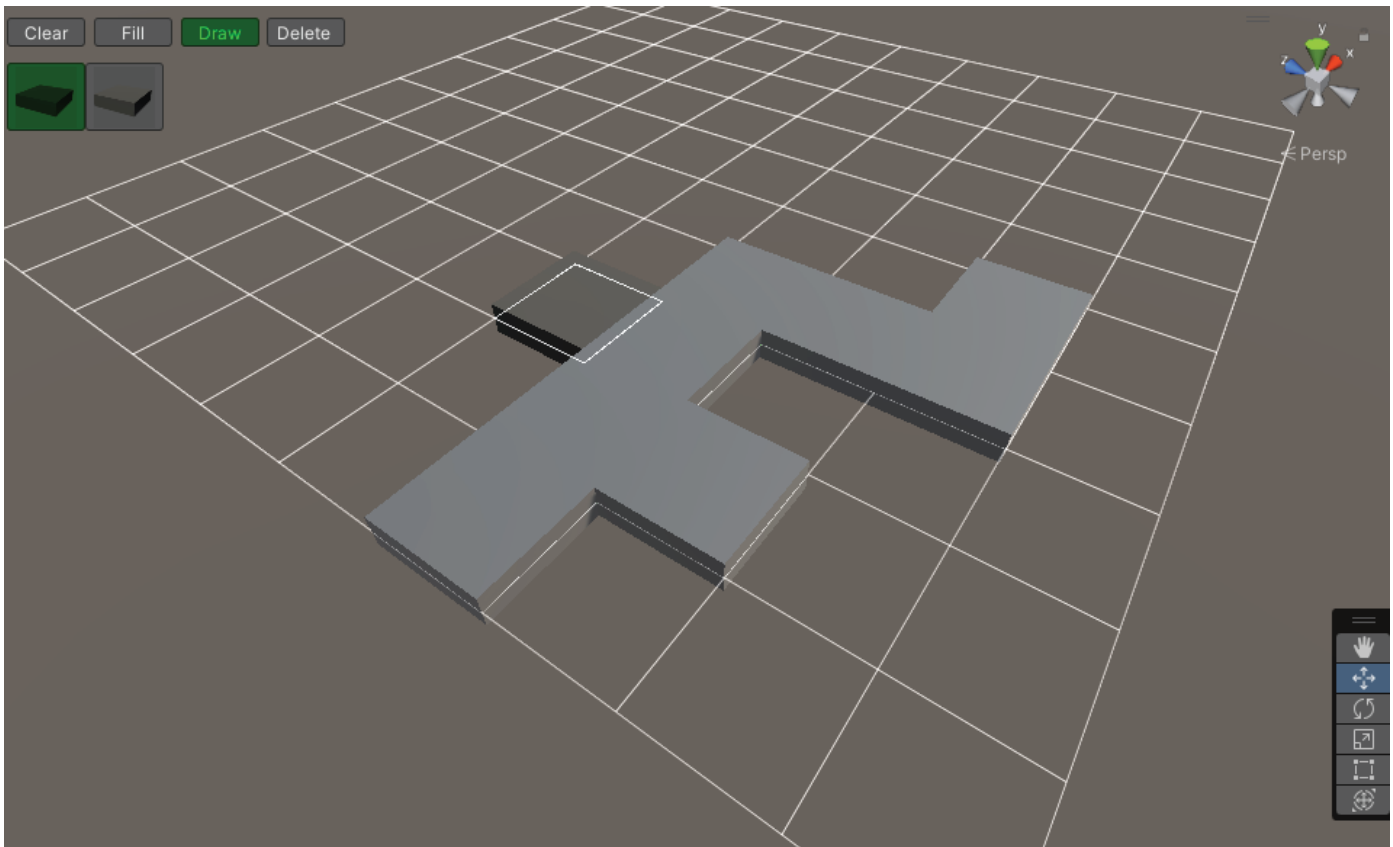


These are the prefabs we created and they are being loaded from the Brushes data asset we created.

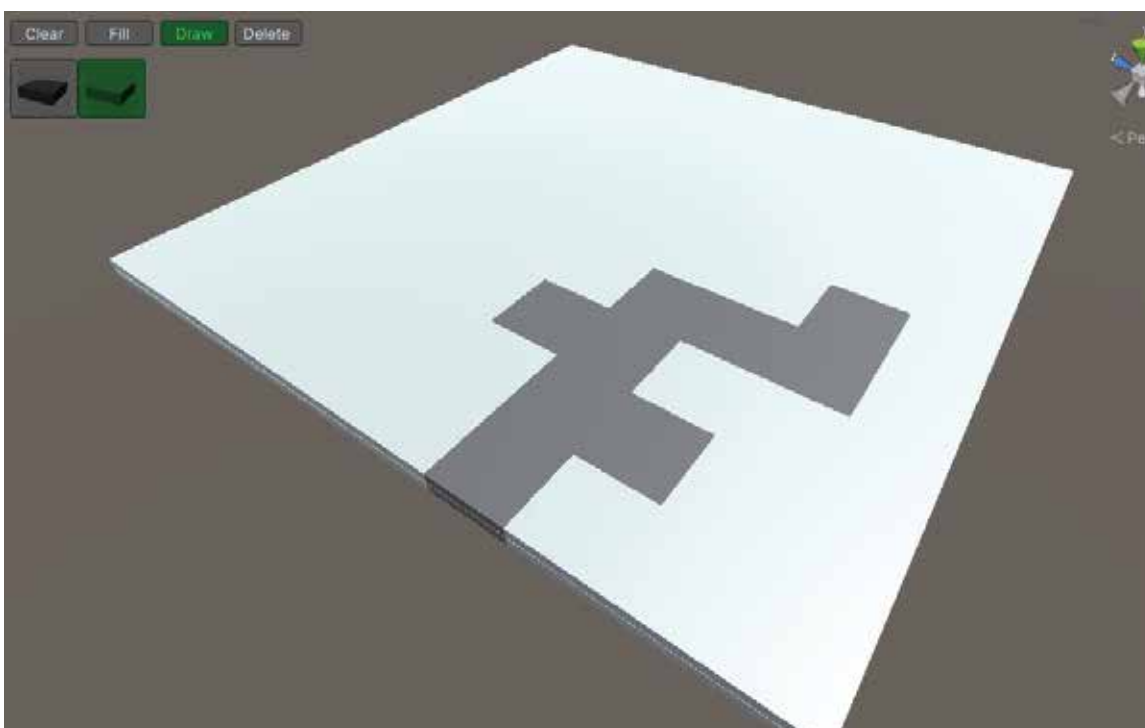


To start painting our grid, select the prefab we want to work with, and the mode we want to use. So any of the prefabs will do, and select Draw just to make it clear.

Using the Grid Builder Tool

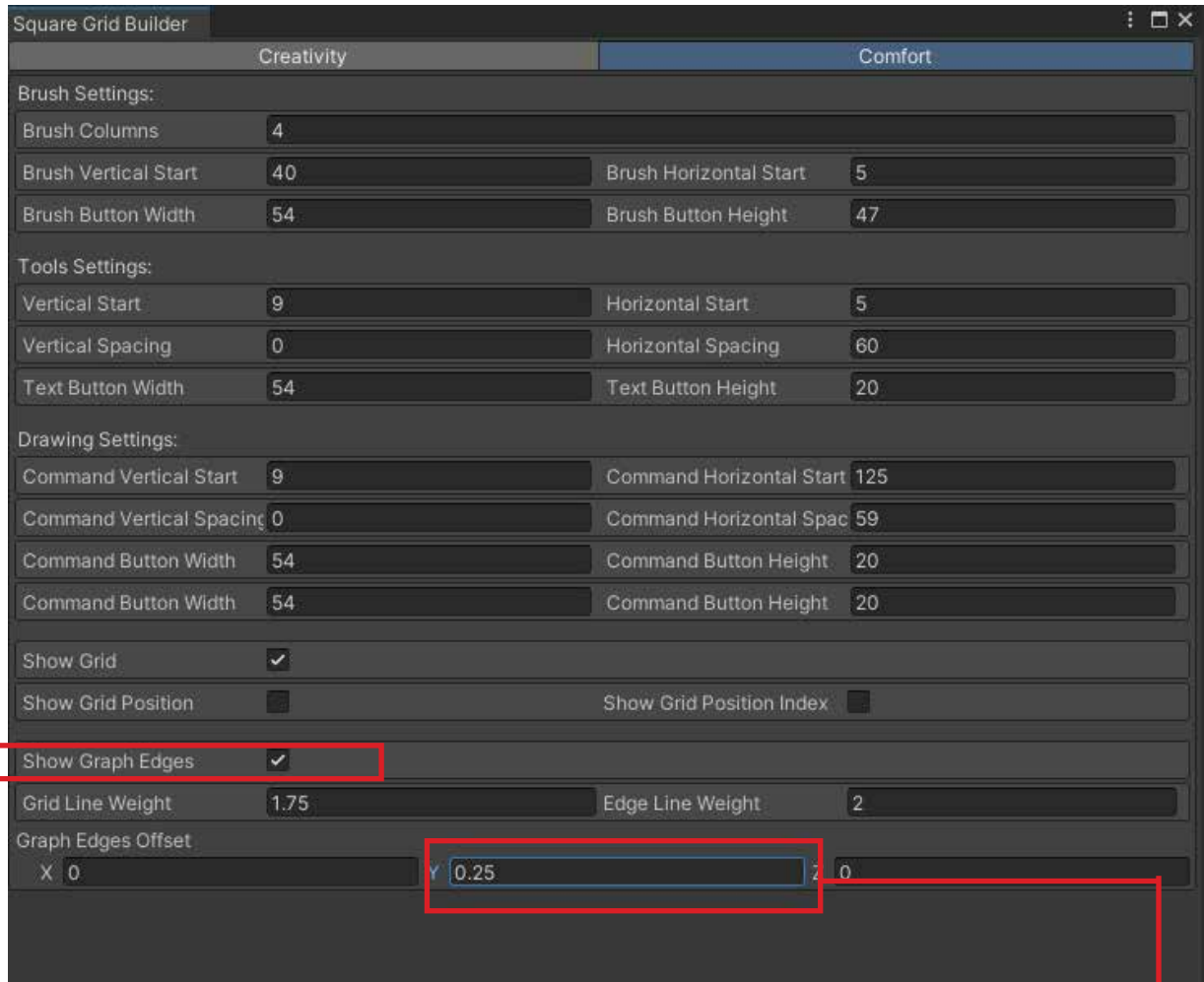


By holding the **Shift** key we can paint our grid the way we want to. To fill the empty grid cells with the currently selected brush, just press fill. If we select Delete we can delete Cells using the **Shift** key



Using the Grid Builder Tool

To visualize what we have done we can select the “Comfort” tab in the Square Grid Builder to open up the configurable settings.

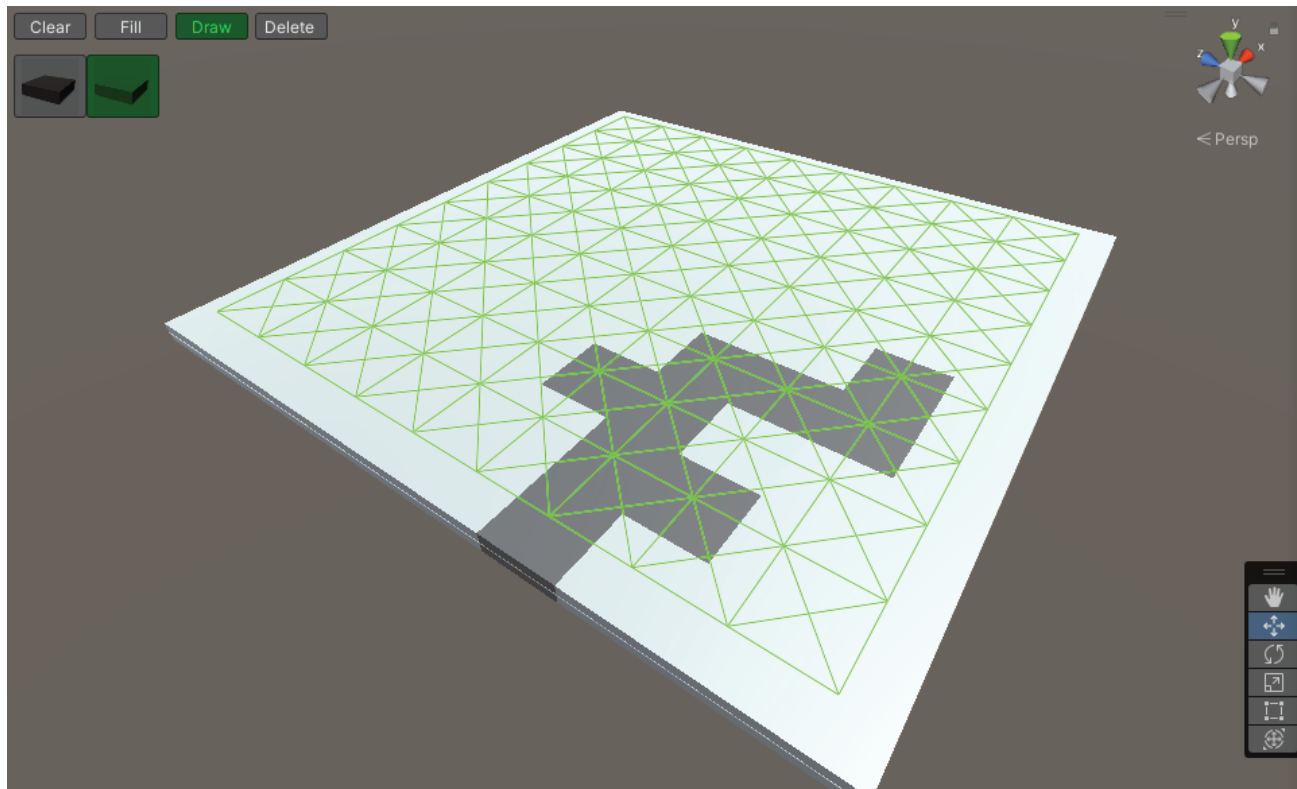


Enable the Show Graph Edges option

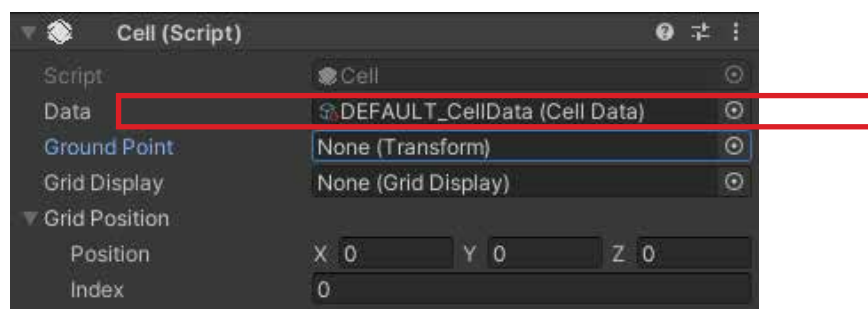
Make sure to adjust the offset a little bit since we are workign in 3D and the cells might cover up our debug visuals

Using the Grid Builder Tool

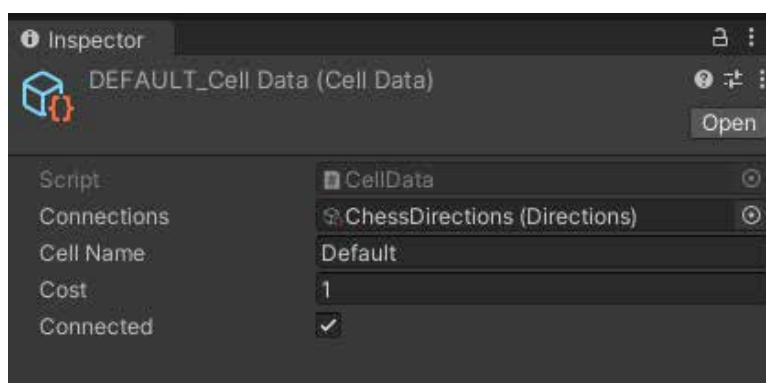
So looking at the grid now we can see the connections between the Cells



Notice that the cells are connected to each other orthogonally and diagonally. This is where the CellData asset comes in, the one that we added to our Cell previously

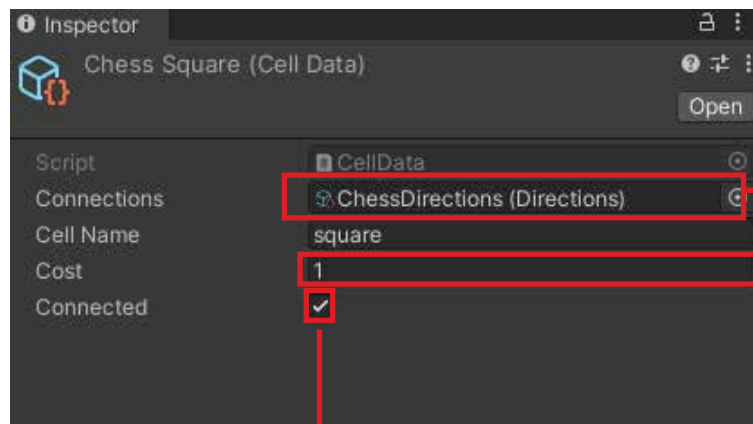


If we click the DEFAULT_CellData(Cell Data) field, and select the file in the project hierarchy we should see this in the inspector



Using the Grid Builder Tool

The CellData asset stores most of the configurable data in the Cell

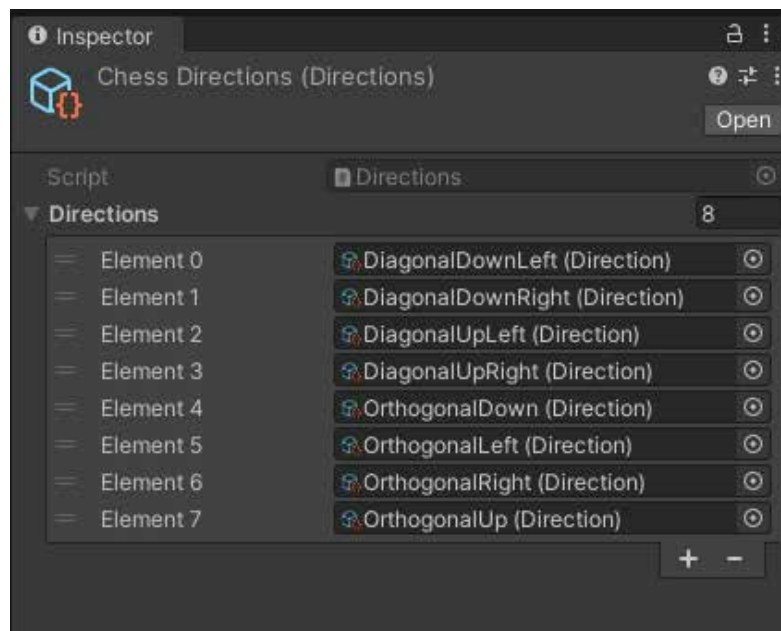


The directions of the Cell. This is where we got those green lines from in the Square Grid Builder

The cost of traversing the cell

If this is false, no connections will be drawn to the Cell. This should be false on Cells that are not at all traversable.

If we click the Connections field (ChessDirections (Directions)) our inspector should look like this

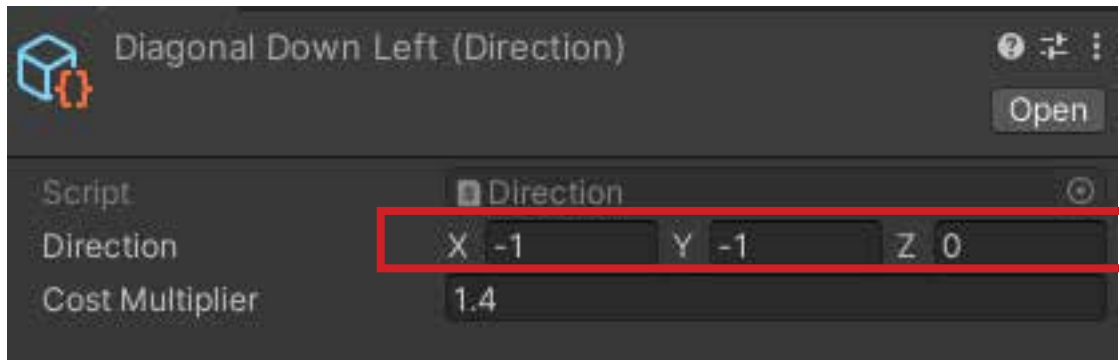


In this asset we can define what directions a cell should be connected in. As we can see here there are 8 directions. Which corresponds to what we saw in the Square Grid Builder

Try clicking on one of the Direction data assets

Using the Grid Builder Tool

This asset tells us specifically how the cell is connected.



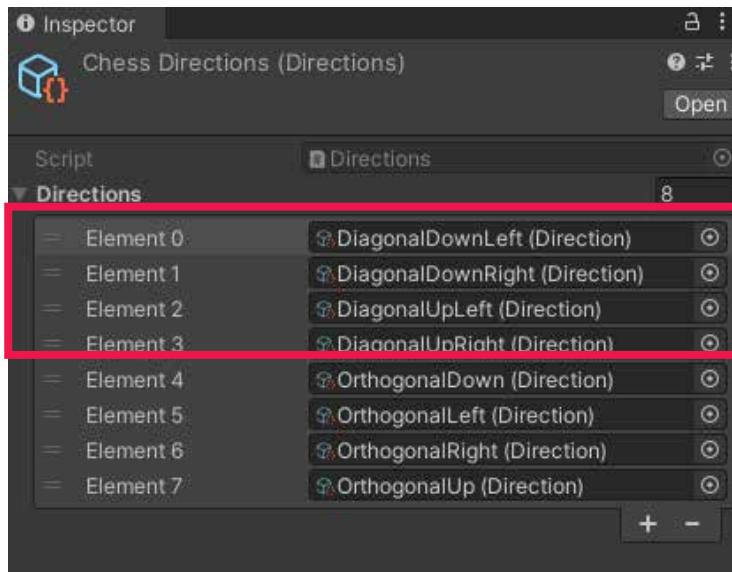
Looking at our Cell at 0,0 we can see that it will produce a connection at position -1, -1 relative to itself. It will also multiply the cost of that Cell by 1.4

1,-1	1,0	1,1
0,-1	0,0	0,1
-1,-1	-1,0	-1,1

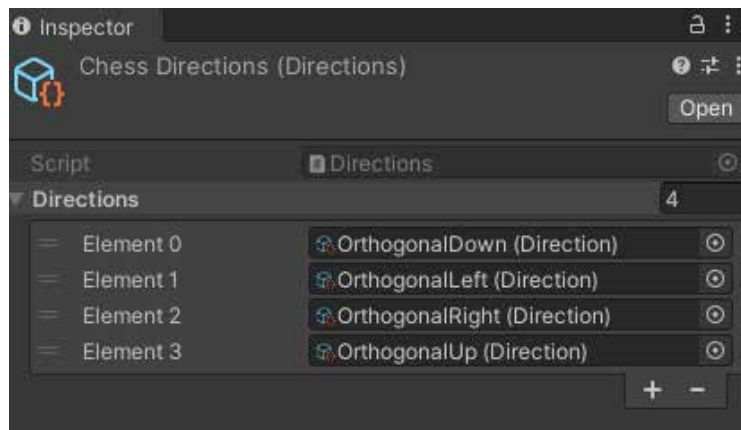
Since we have 8 different directions in the Directions data asset, it probably means the Cell is connected both orthogonally and diagonally to it's closest neighbours.

Using the Grid Builder Tool

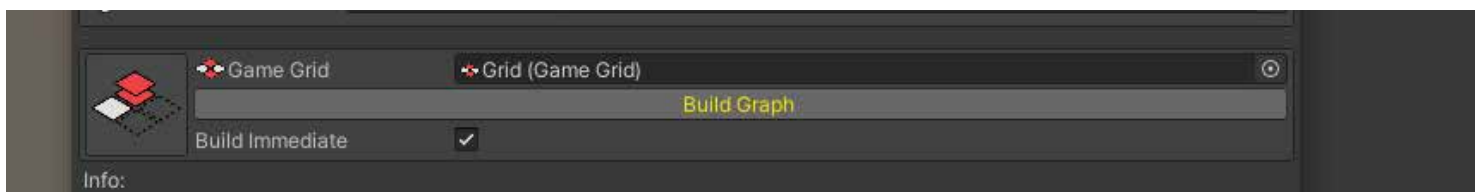
If we go back to the ChessDirections asset



And remove all directions beginning with 'Diagonal' It will look like this:

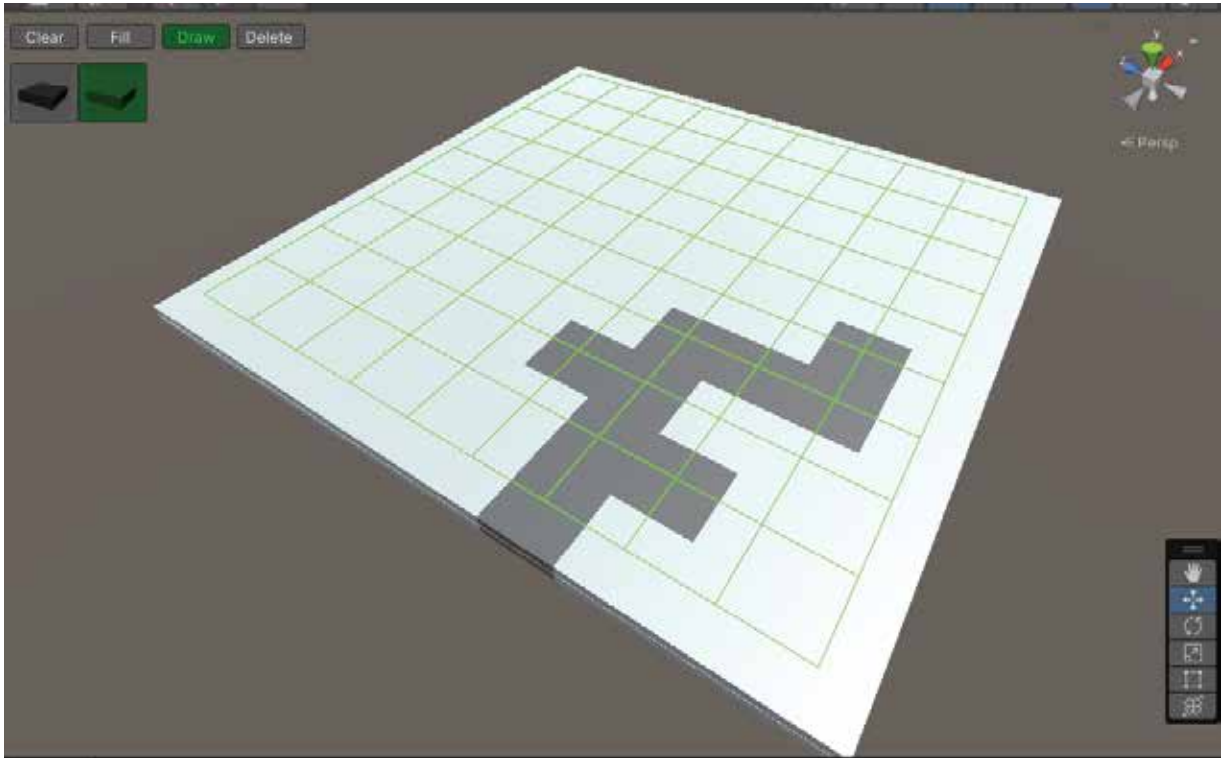


We can now go back to the Square Grid Builder - If it was left open while doing all of this we can simply press the button "Build Graph"



Using the Grid Builder Tool

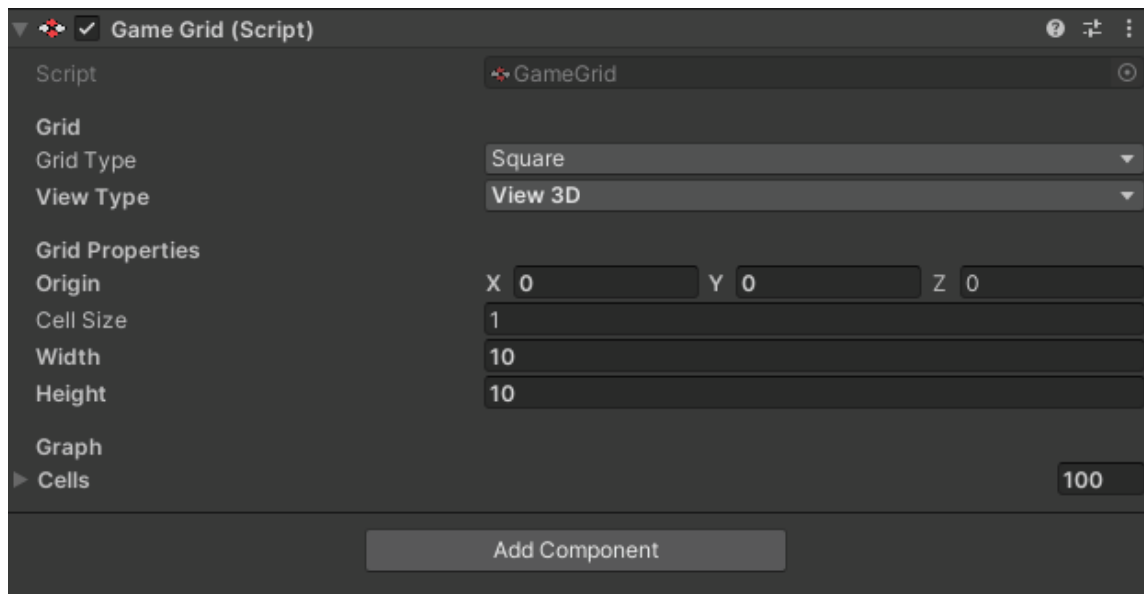
Our grid now only has 4 directions per cell, and the resulting graph will be connected like this instead:



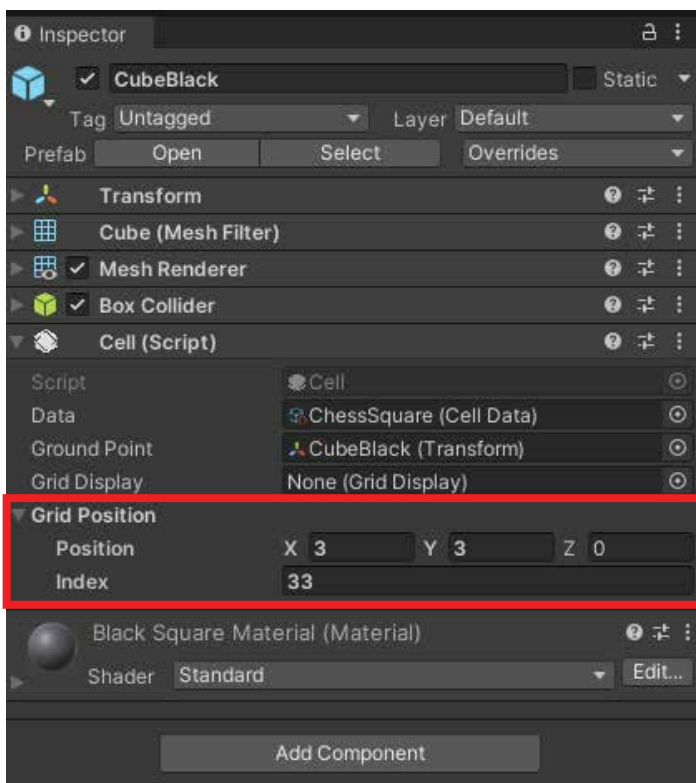
In practice this means that we can build any type of graph using the CellData assets and this gives us a lot of flexibility when designing new games because it's fast and easy to create new movement patterns, and different cell types like Hexagonal and Square cells can share the same data.

Using the Grid Builder Tool

If we look at the Grid object again we can see that the properties have been updated



Also, looking at one of the Cell objects we can see that it has some new values in the Grid Position field.



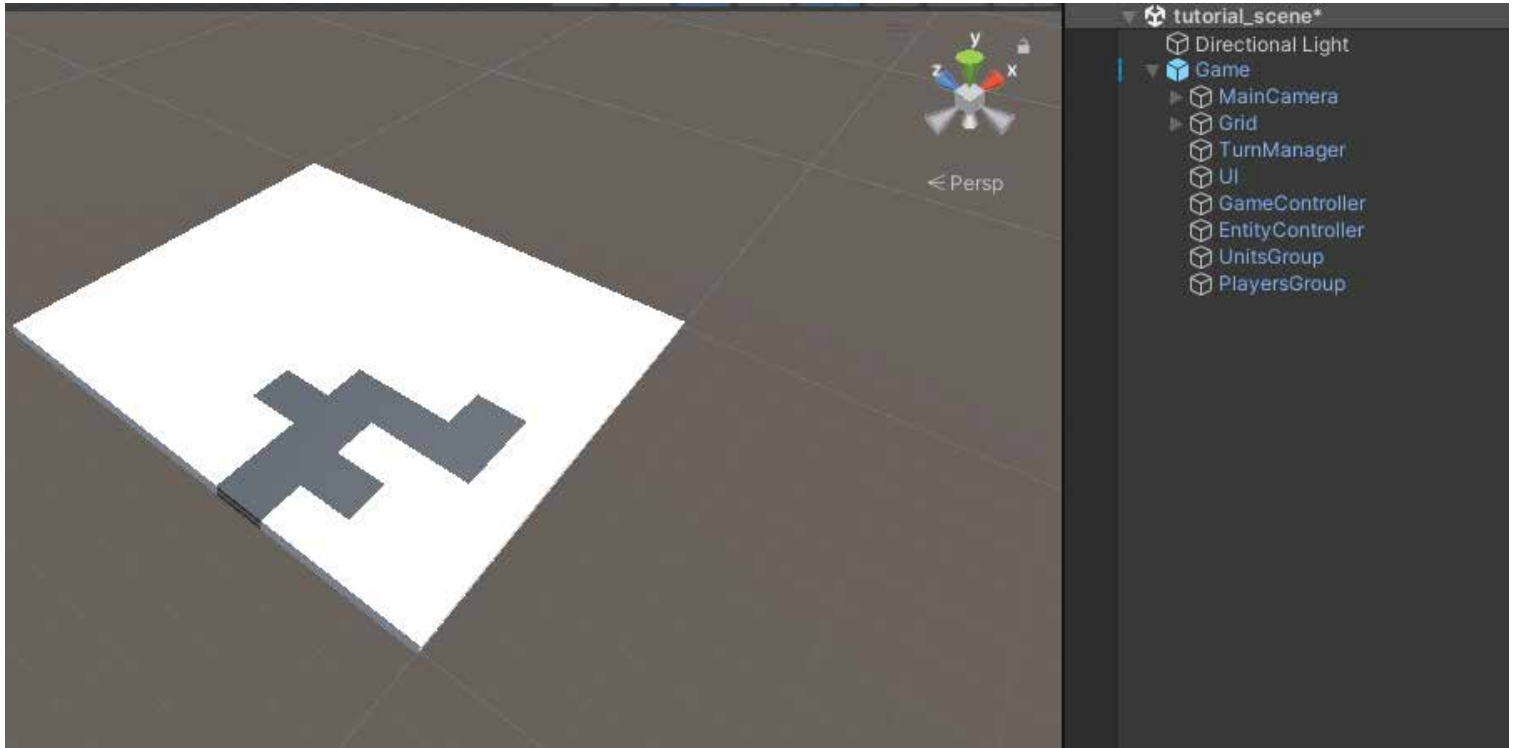
The Grid Position is one of the most important fields in the Cell. It is used to calculate the directions, fetching the Cell from it's index value, used in pathfinding and so on.

So if we want to create your own grid without using the Grid Builder tool, we will have to set it manually, and make sure it follow the correct pattern, and have no identical indicies.

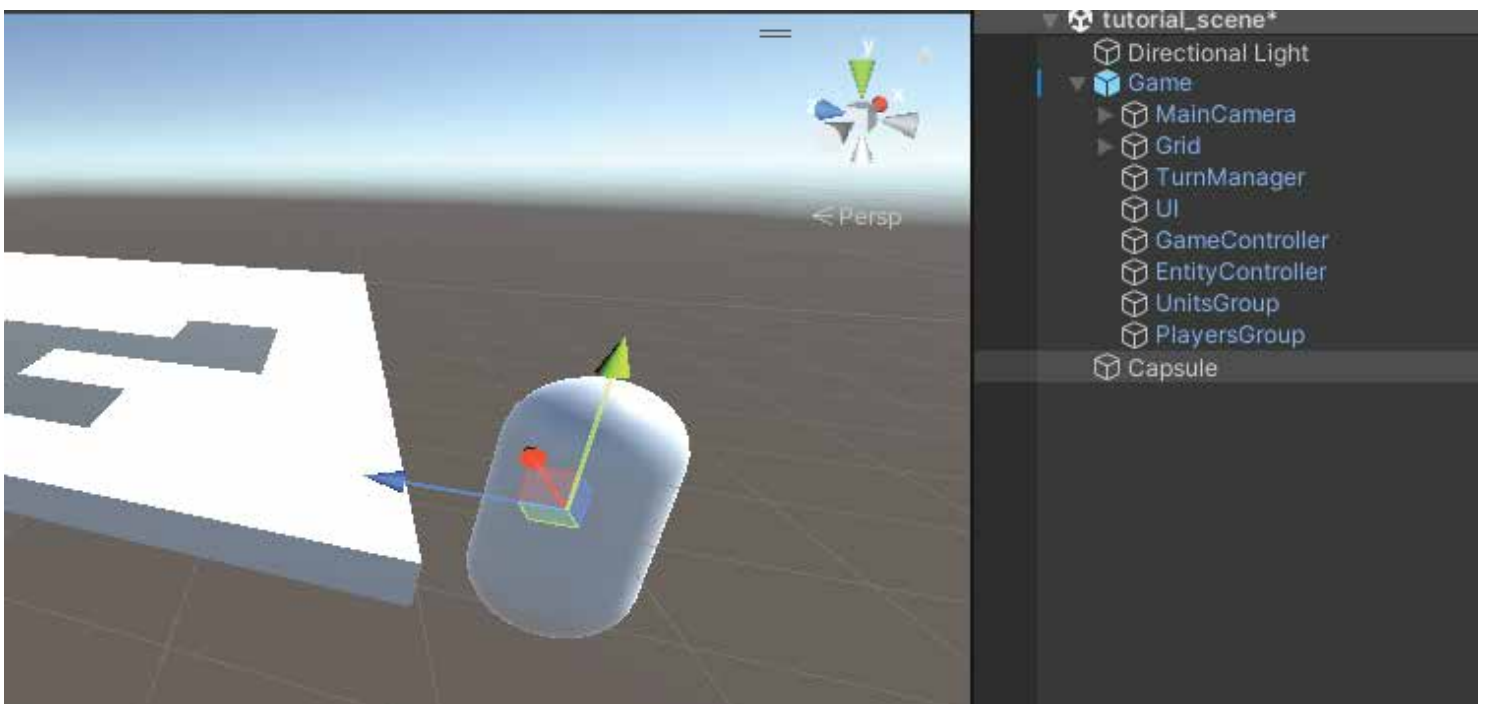
Creating Units

Lets go ahead and create a unit that can move on the grid.

Our scene should look something like this now

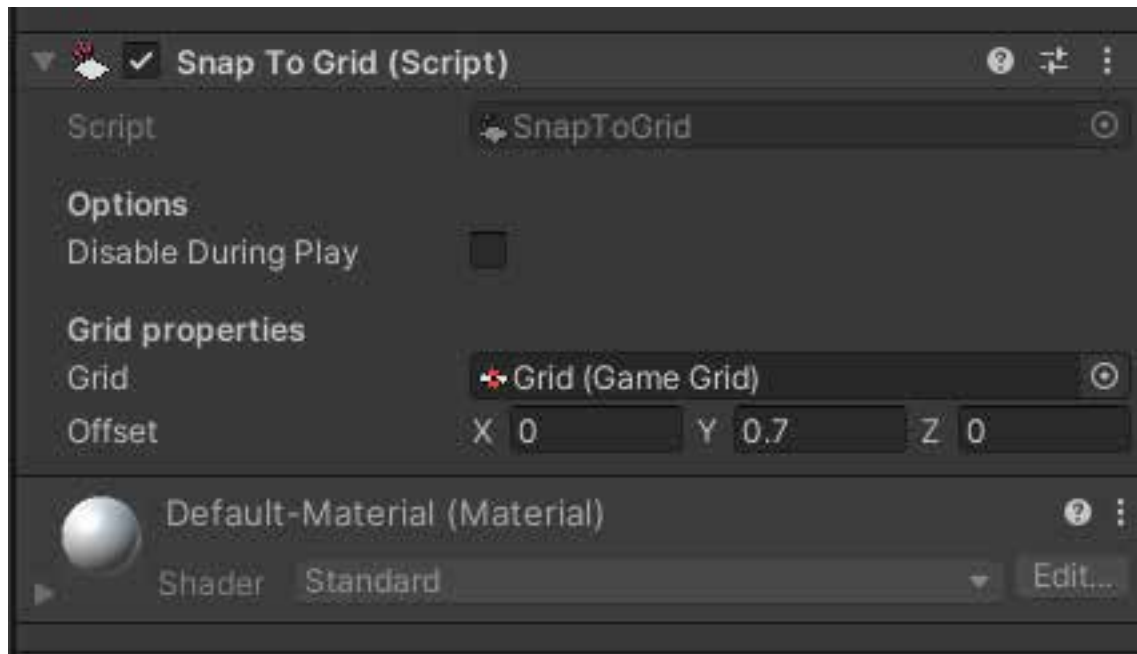


Create a capsule primitive in the scene hierarchy, set the position to 0,0,0 and scale it to an appropriate value. Make sure that the capsule has a collider with the "Is Trigger" flag set to true



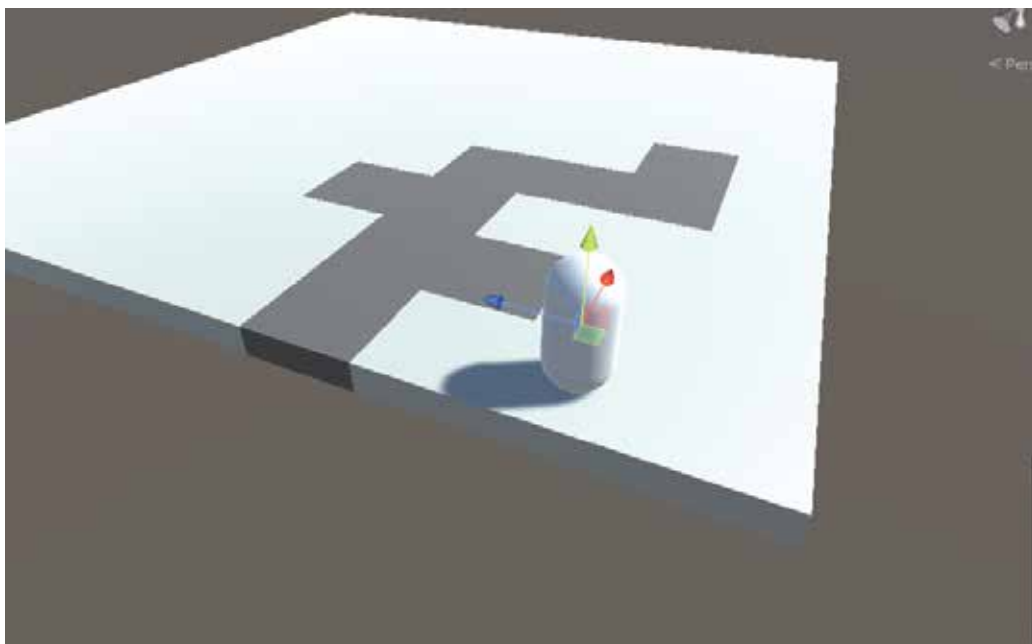
Creating Units

To make life easier for us, we can also attach the Snap To Grid Component, and specify our grid in the Grid field.



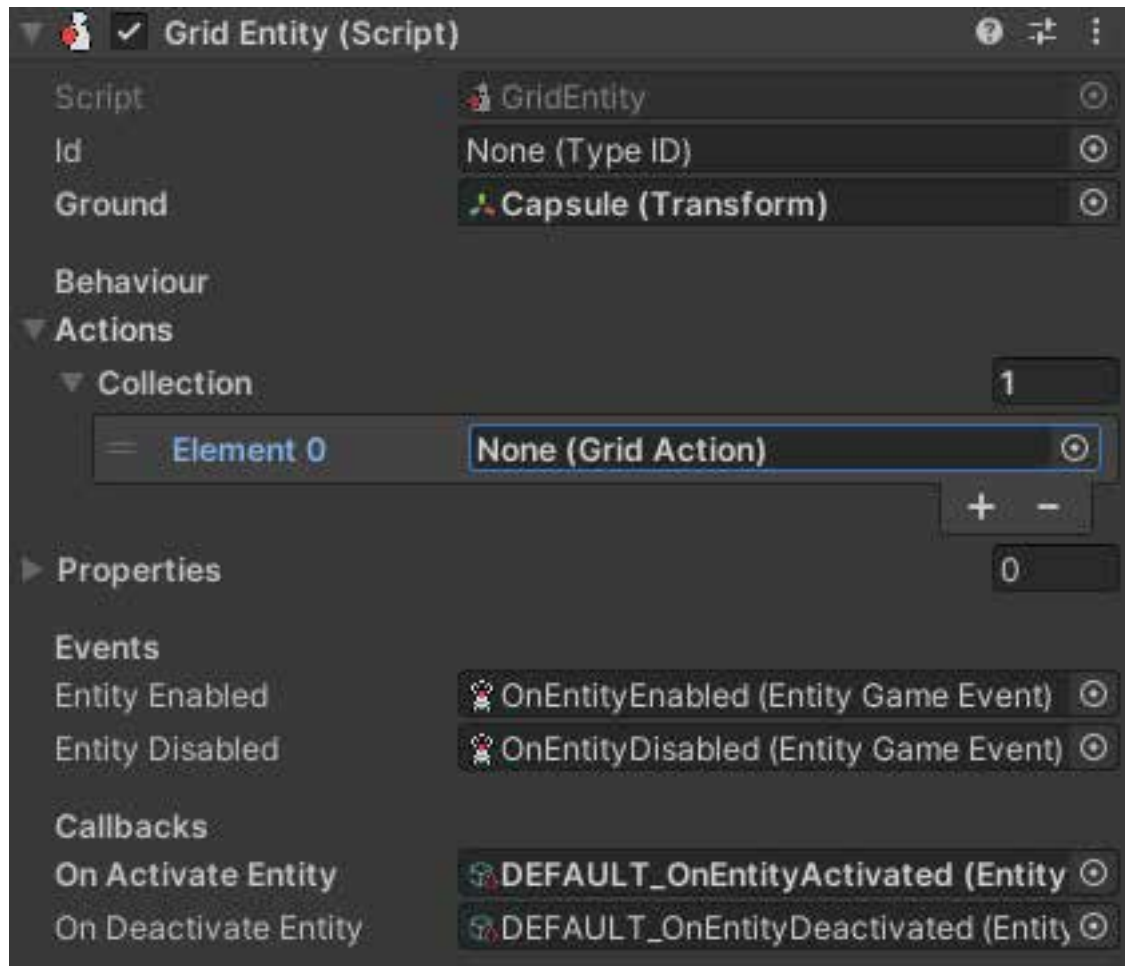
Use an offset to make the unit stand in the correct place

Now that we move the capsule around, it will position itself in the center of the grid cells.



Creating Units

Now we can start making a behaviour. Attach a GridEntity component to the capsule. This component is responsible for controlling the unit, calculating it's current Cell and it provides states to the game runner from the Actions field. The GridEntity component requires one or more GridActions to actually output any behaviour.

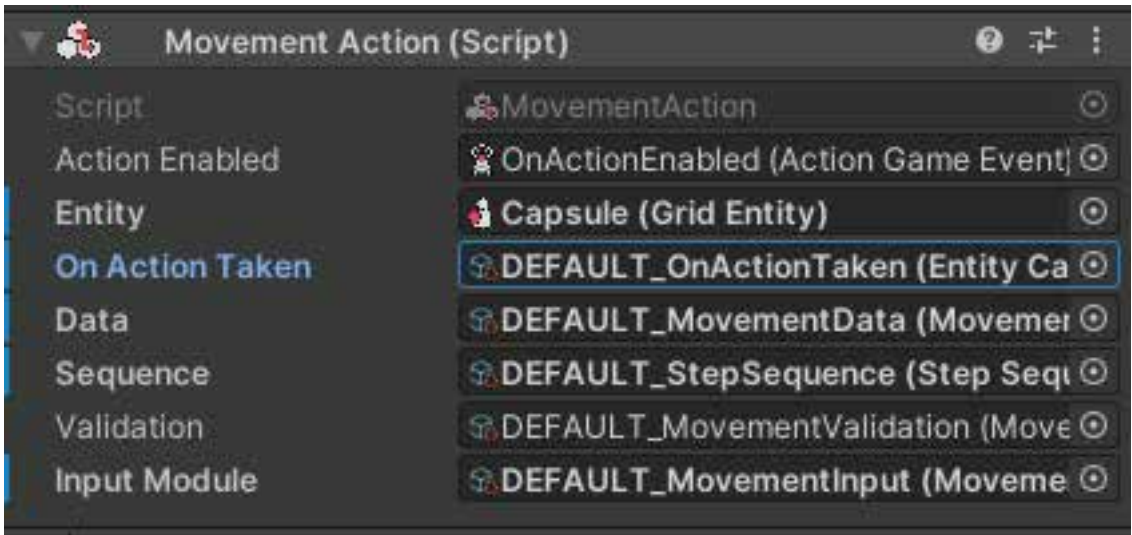


The only field we have to care about right now is the Ground field. Just drag the capsule transform to the Ground field.

Creating Units

Specify the GridEntity to be GridEntity component that we attached to the capsule. If this field is left empty, the script will search for it using the GetComponent method.

In the other fields let's use all the default values for now.



To get things to work we need to look at this specific field called Input Module

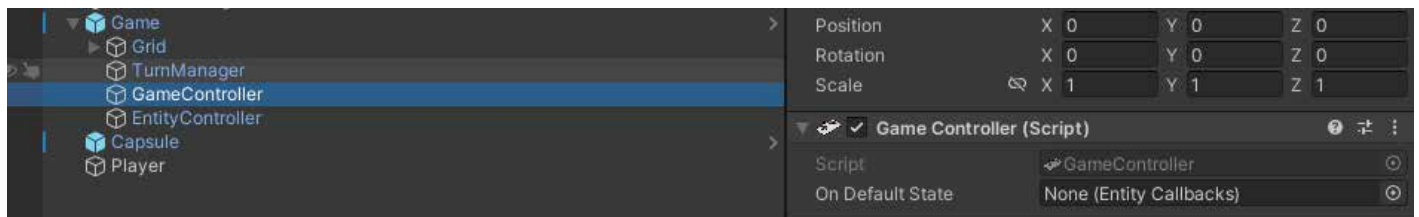
Creating Units

The Input Module provides a state when a GridAction is selected and allows us to configure behaviour for a GridAction.



The one important thing to remember is that Input Modules need to be initialized by the Initializer to get all its dependencies. By default, all Input Modules are initialized by the Initializer by searching the project's assets.

To get a better understanding of how this works, let's look at the GameController object in the Game prefab in our scene.



The GameController is responsible for running the game loop. It does this using the state pattern. The states will define what happens in the game at any given moment - and will return a new 'next state' when they have reached a conclusion.

```
void Update()
{
    if (_on)
        Progress();
}

///<summary>
/// Runs the current state until null is returned - at which point the default state will run.
///</summary>
/// Frequently called (5) 1 usage 1 calls
private void Progress()
{
    _currentState ??= DefaultState;
    _currentState = _currentState.Run();
}
```

This is what the Update looks like in the GameController

Creating Units

This is where the InputModule comes in to play. It provides a State to the GameController (We will look at how later..) And by creating new Input Modules we can have new behaviour even though the GridAction components are shared.

Coming back to our Movement Input Module

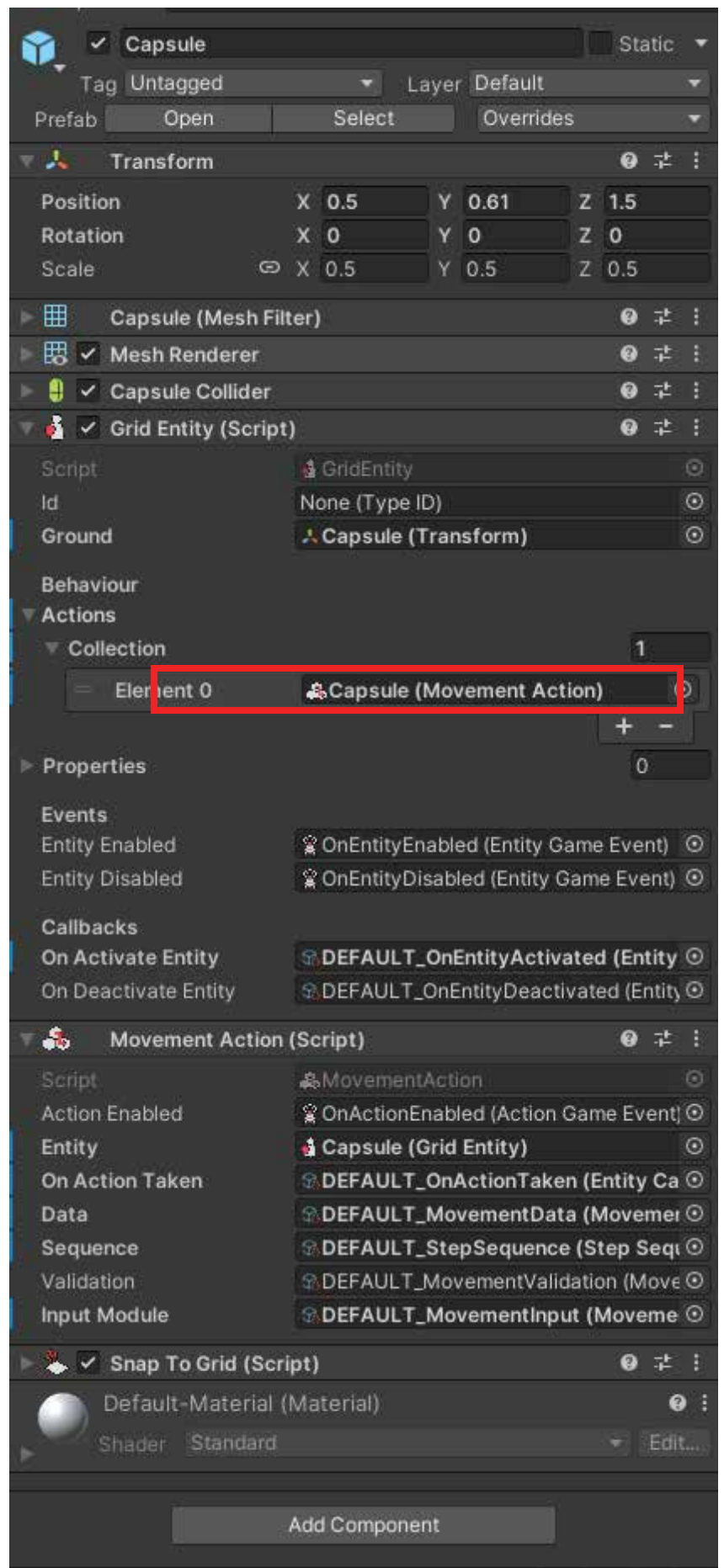


This module in particular will return a state called ConfirmMovementState which will allow the player to move a GridEntity on the grid. To be able to do this it requires a few helper classes. Any class which inherits the InputModule will have the Initialize Method provided.

By default the Input Modules are initialized by the Initializer by searching the assets. You can of course limit this search to a particular folder, or create your own custom list of Input Modules to be initialized.

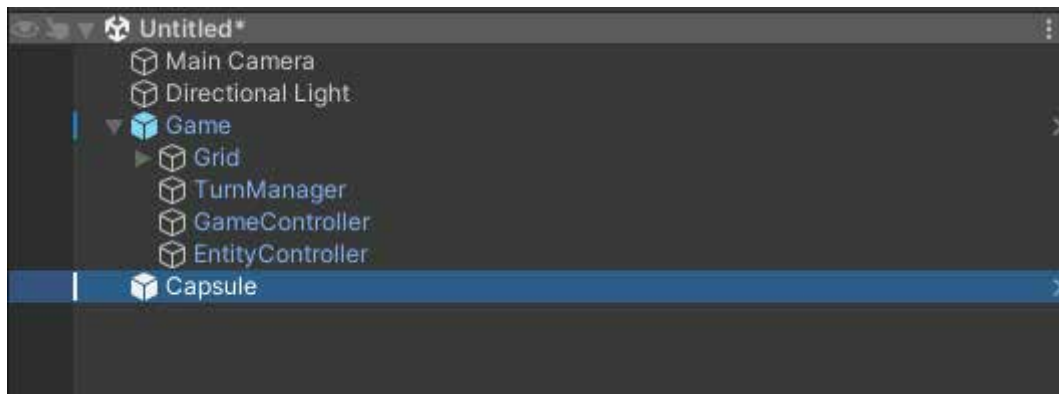
Creating Units

Go back to the Capsule object and make sure the MovementAction is added to the list of Actions on the GridEntity component



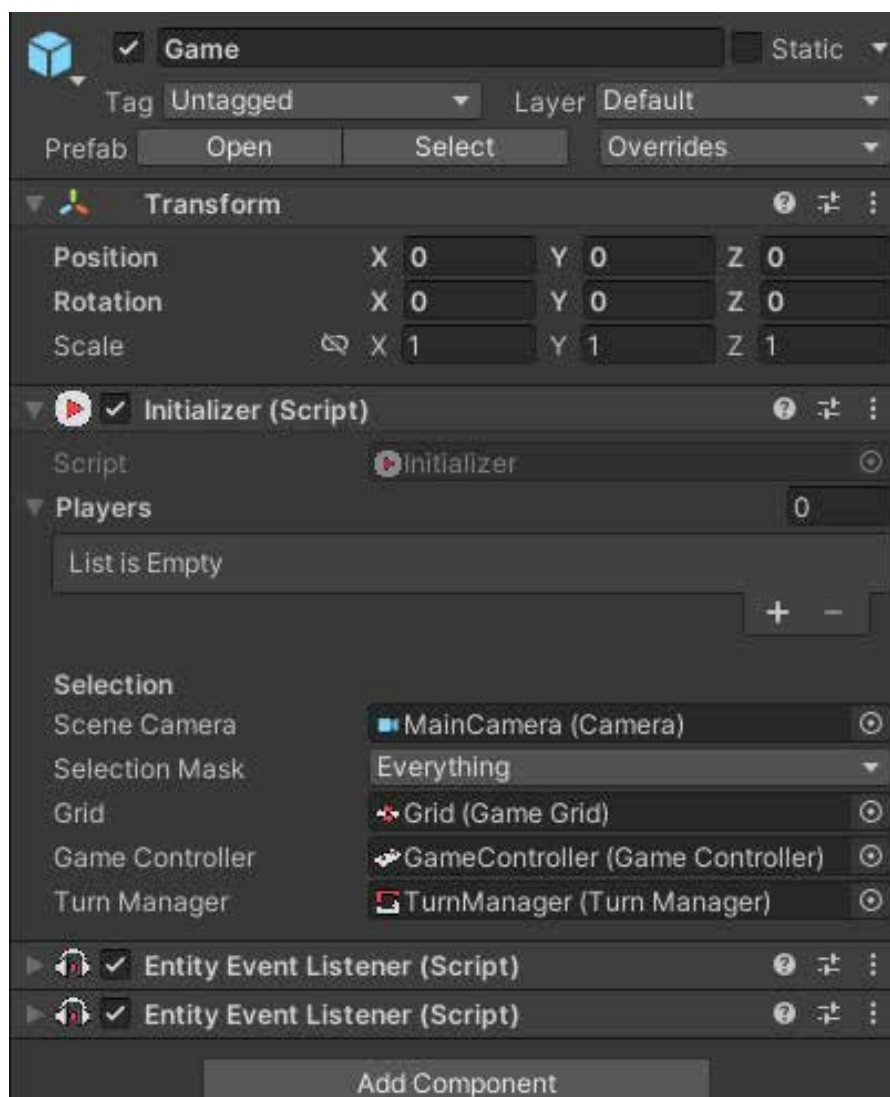
Creating Units

The hierarchy should now look something like this



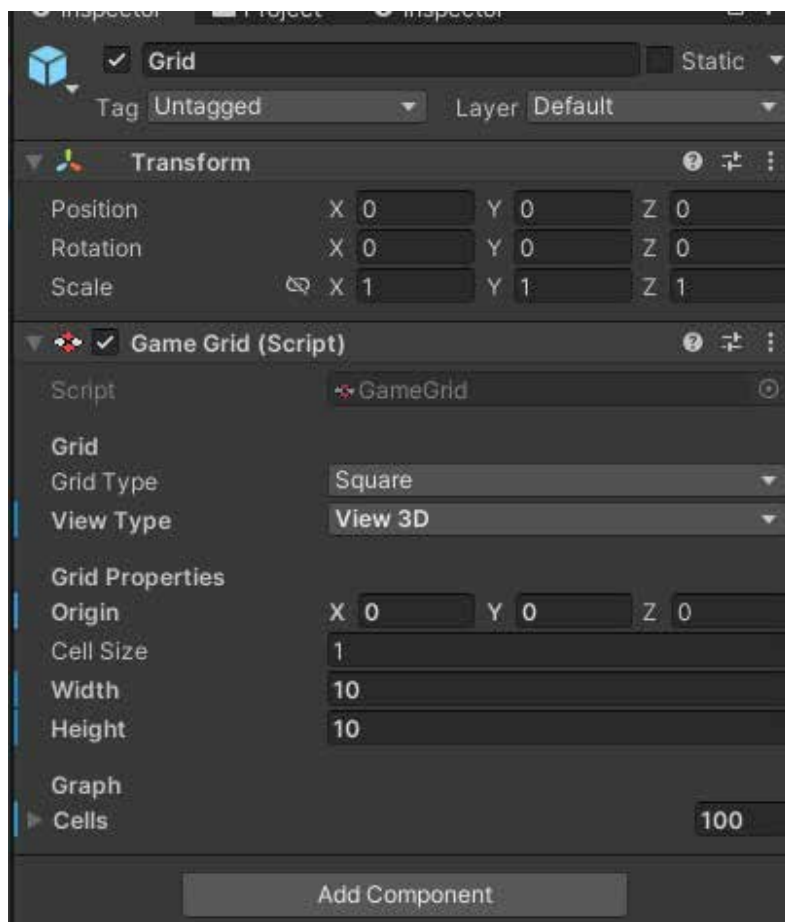
And all the controller in the Game prefab:

Game

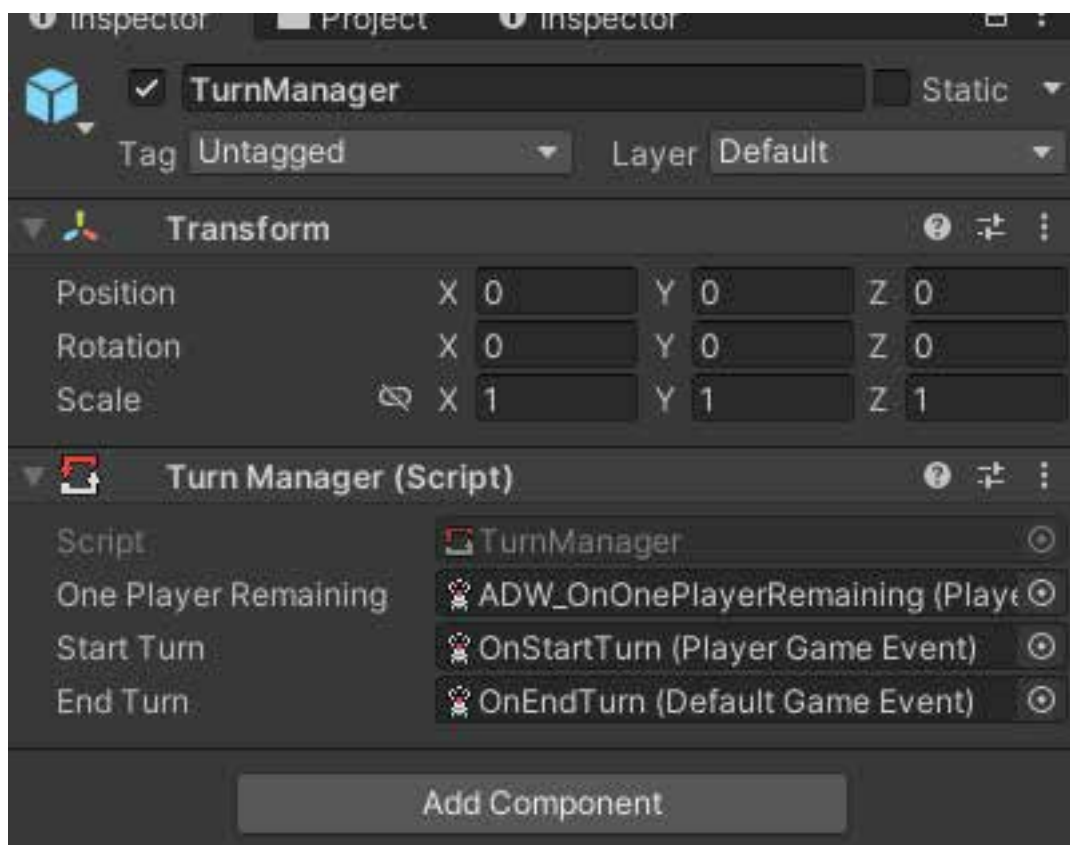


Creating Units

Grid

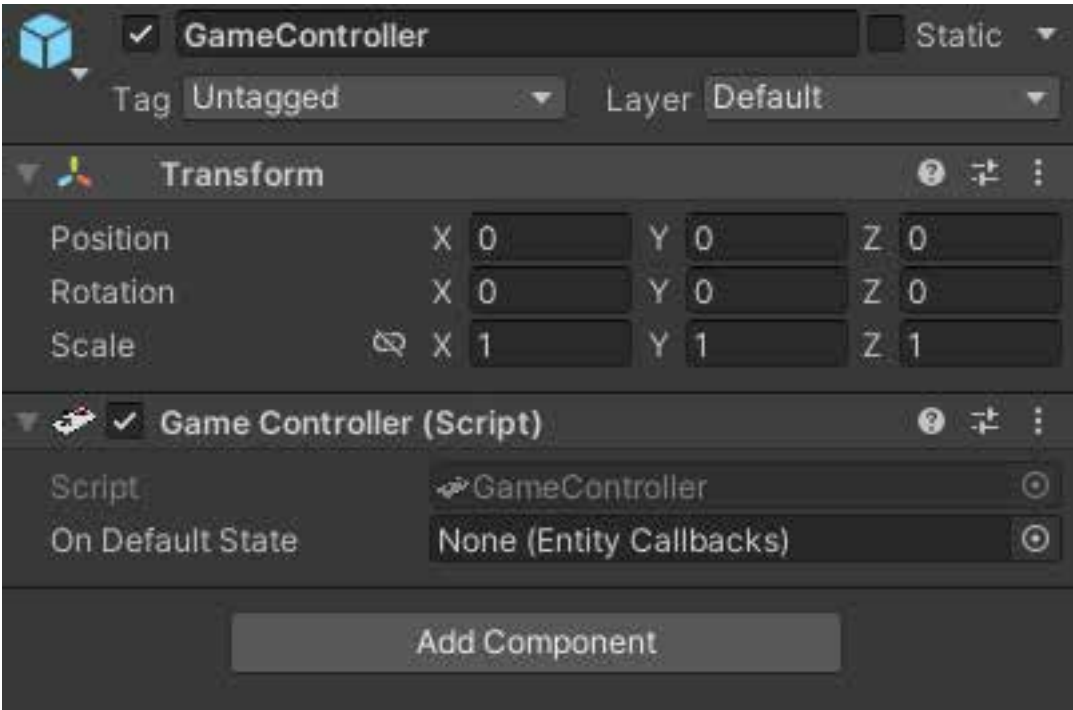


TurnManager

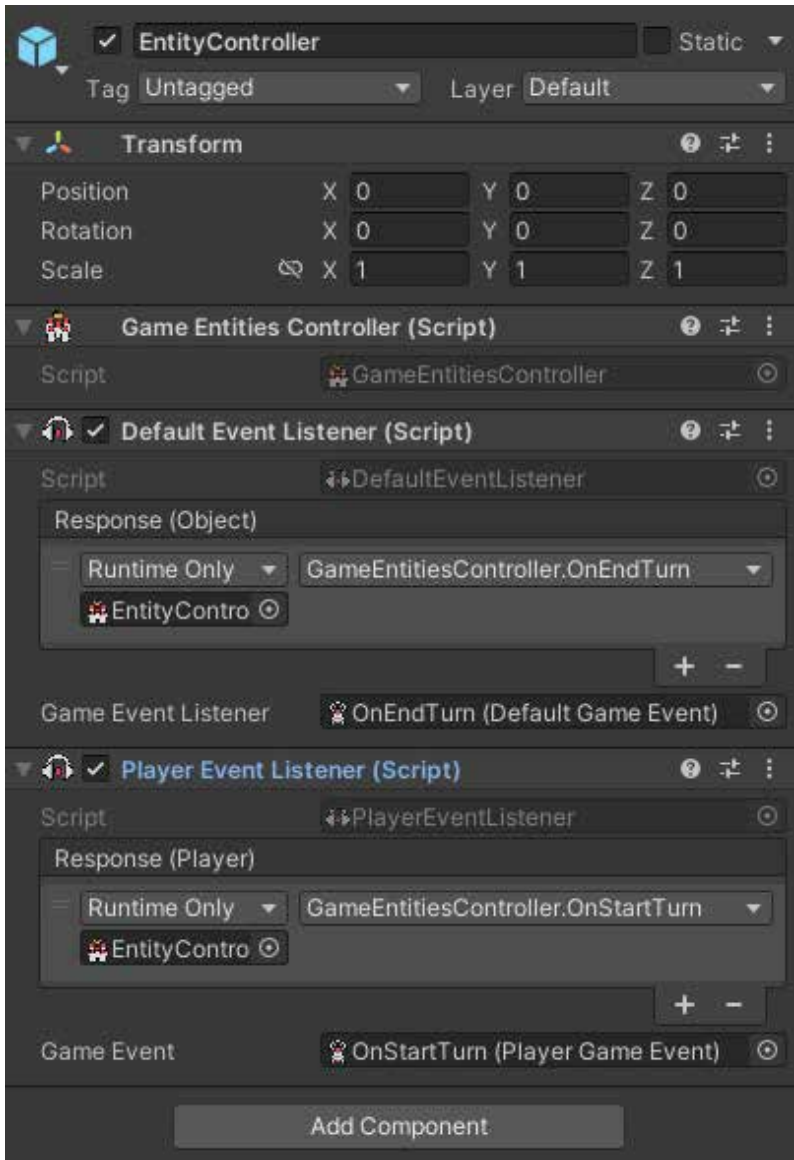


Creating Units

GameController



EntityController



Creating Units

Try left-clicking the capsule on the grid and this error might appear.

A screenshot of a Unity console error message. It shows a red error icon, a timestamp [10:41:57], the exception type NullReferenceException, the message Object reference not set to an instance of an object, and the file path Gridr.Gameplay.RaycastListener3D`1[T].IsHitUI (T& selectedUI) (at Assets/Gridr/Runtime/Scripts/Gameplay/Selection/RaycastListener3D.cs:53).

[10:41:57] NullReferenceException: Object reference not set to an instance of an object
Gridr.Gameplay.RaycastListener3D`1[T].IsHitUI (T& selectedUI) (at Assets/Gridr/Runtime/Scripts/Gameplay/Selection/RaycastListener3D.cs:53)

The default selection handler (called IListener) uses Raycasting to detect object in the game. So we need to have an Event System in our scene.

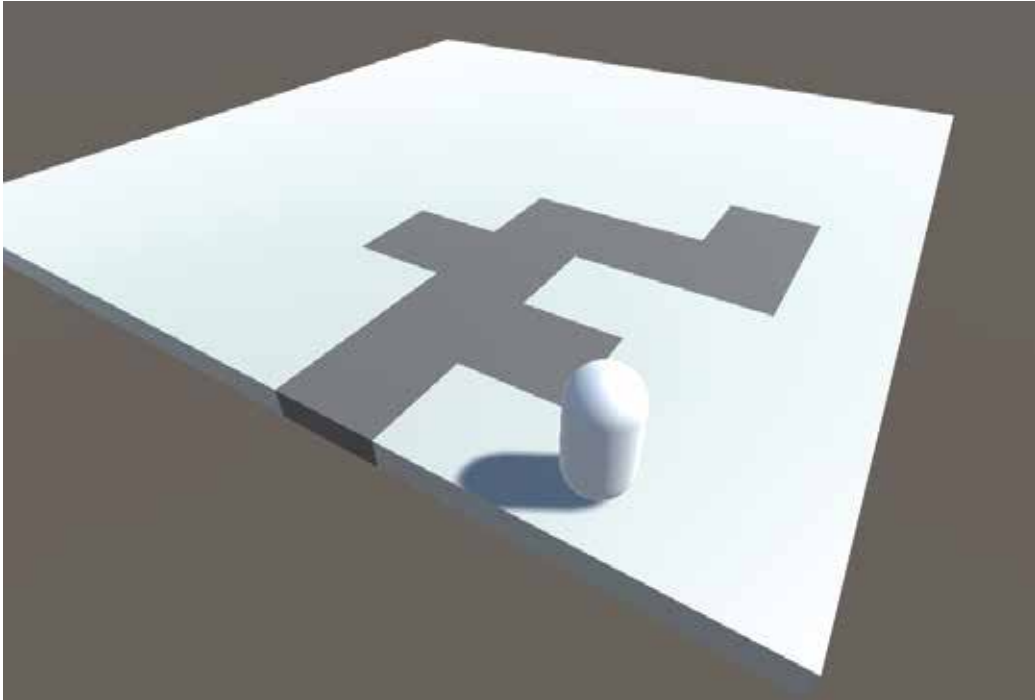
In the scene we can right-click and select UI -> Event System. This should be included in the Game prefab, as a child of the MainCamera



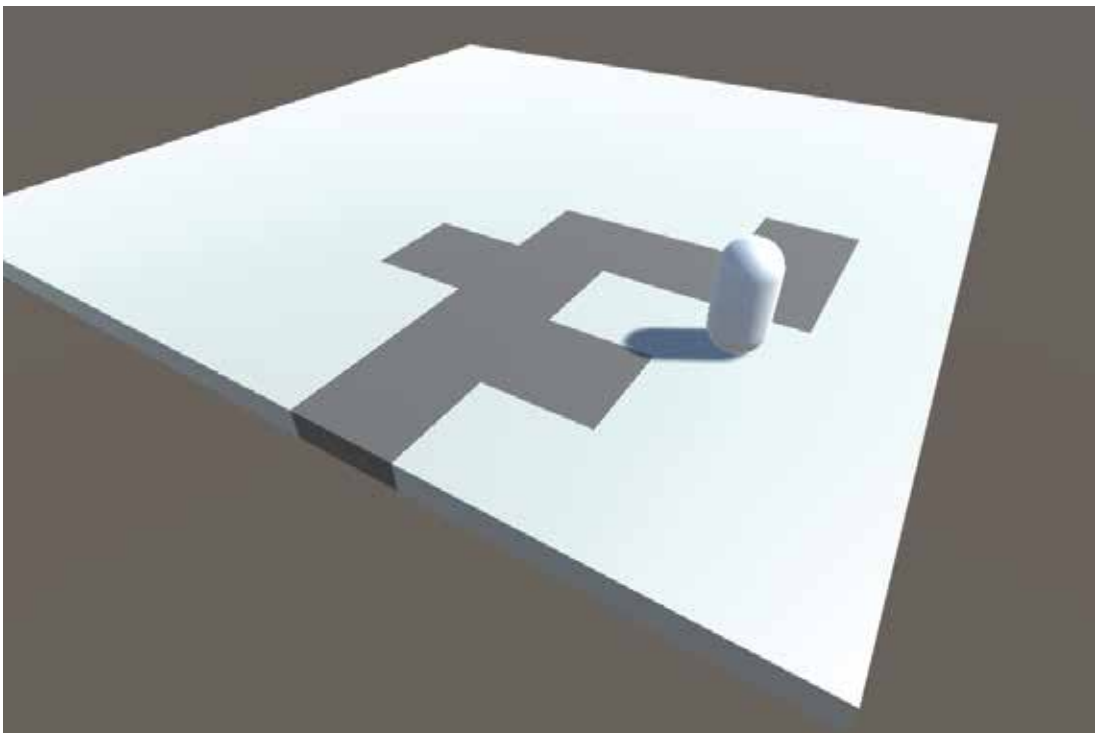
Creating Units

Hit play again and left-click the capsule once. Nothing will happen, but after having clicked the capsule once, try clicking one of the nearby cells.

*If you get an error saying that a key has already been added in the MovementAction script, open up the Square Grid Builder once, and then close it again. This will re-build the grid and usually solves any problems related to duplicated grid positions or cells.



The unit will move cell by cell to the clicked cell provided it is close enough

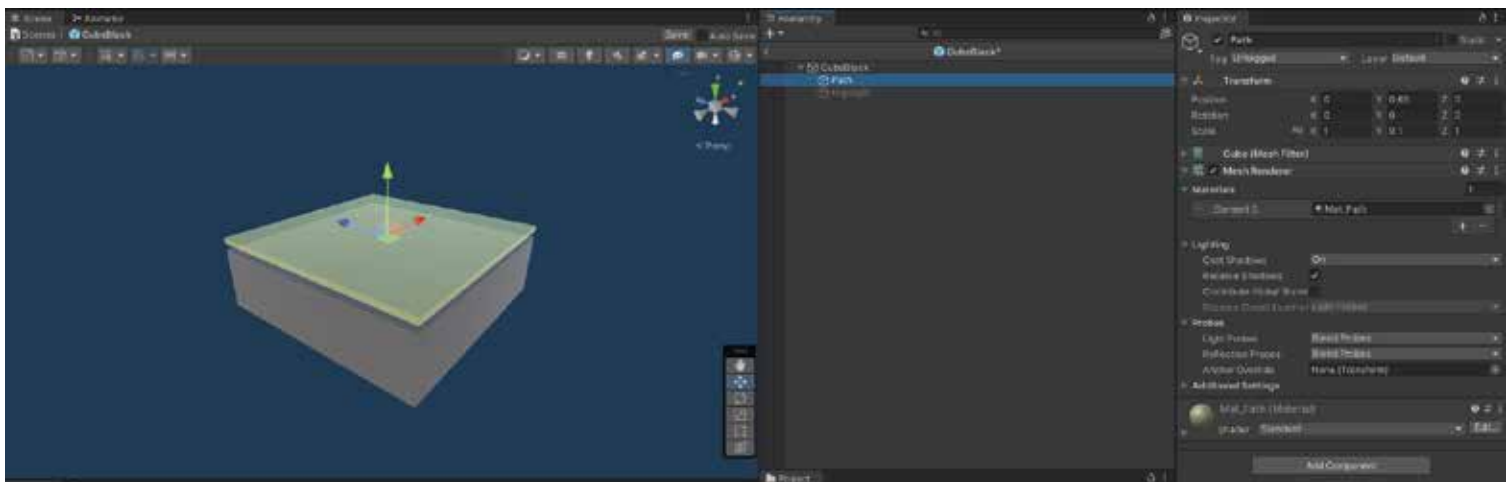


Creating Units

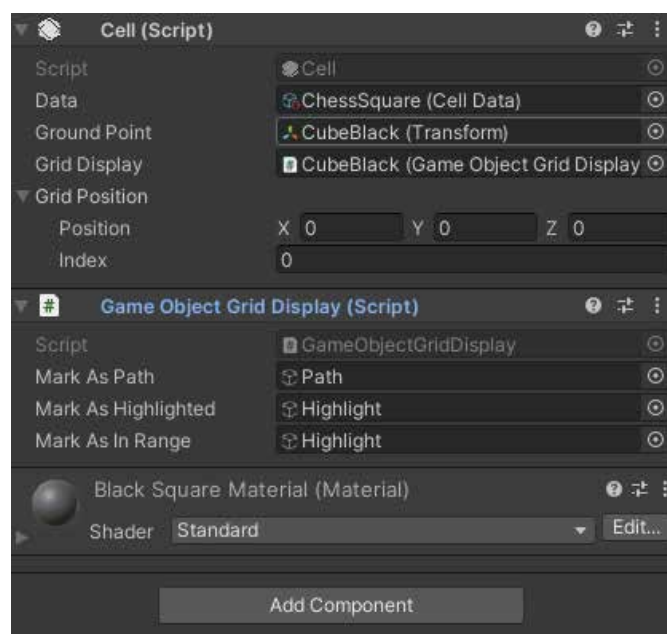
Ok so we are almost there! It works but we can't see anything on the grid. Let's fix that.

In the Cells, add a couple of game objects to act as our UI. Here I used a 3D cube and added the Mat_Path material, as well as one with the Mat_Green material. I also adjusted the y-scale and moved it's position to be on top of the Cell. Also leave them turned off as default.

** Make sure no colliders are added on these objects! It will mess up detecting the underlying Cell.

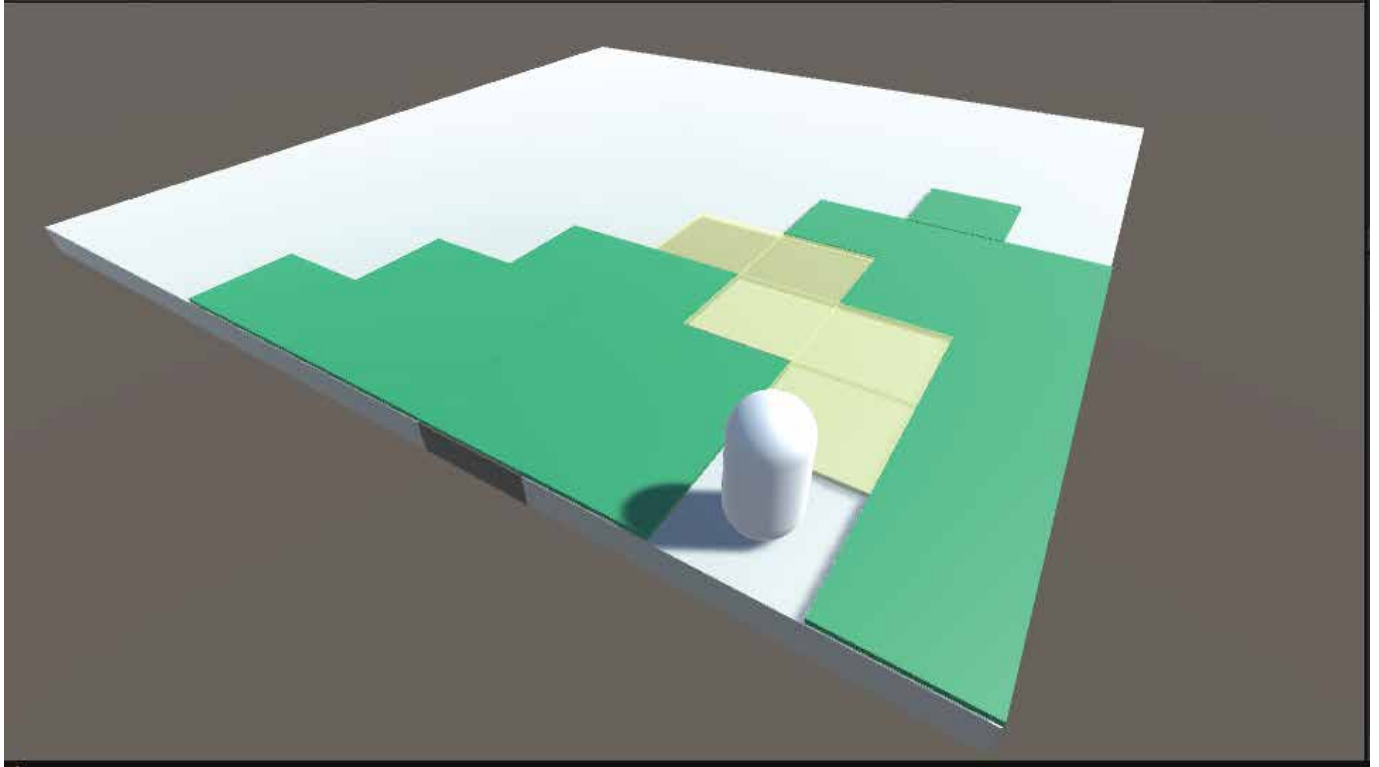


Add the Game Object Grid Display component to the Cell, and make sure to drag the Grid Display component to the Grid Display field in the Cell. Then add the game objects we just created to the Mark As Path and Mark As Highlighted fields. Do this for both cells.



Creating Units

Now if we click play and select the capsule again we should see that the movement range is visible in green, and the path that the capsule will walk is marked in yellow!



How does it run?

Now that we have created a Grid and a Unit that can walk the grid. Let's see how it all works.

The Game prefab in our hierarchy has a component called Initializer. That is the entry point of the game.

If we look inside the Initializer script we see that during Awake, it will Initialize all the objects that need to be initialized. What does this mean? There are a few necessary objects that we need to be able to run the game properly - the GameGrid, IInputReader, IListener<ISelectable> and IActionInputSettings.

We already know what the GameGrid is. It is the grid of cells that we created in the first part of the tutorial.

The IInputReader is an interface that is used to detect basic user input - like primary or secondary button triggered, and where the pointer is. By default, a MouseInputReader is created to implement this interface.



```
///<summary>
/// Creates the input reader to be used during the game.
///</summary>
///</summary>
private void InitInputReader()
{
    _inputReader = grid.viewType switch
    {
        ViewType.View2D => new MouseInputReader(sceneCamera, Vector3.forward),
        ViewType.View3D => new MouseInputReader(sceneCamera, Vector3.up),
        _ => _inputReader
    };
}
```

This class is simply a wrapper class on top of the UnityEngine.Input input class. You can change this to any implementation of your liking. But what it means is that when the game is listening for input, it will use IInputReader.Primary() to determine if the primary input button was triggered..

How does it run?

Now on to the `IListener<ISelectable>`

Initializing the `IListener` looks like this

```
///<summary>
/// Creates a listener and fallback listener depending on grid type
///</summary>
///</summary>
private void InitSelectableListener()
{
    _listener = grid.viewType switch
    {
        ViewType.View2D => new RaycastListener2D<ISelectable>(sceneCamera, selectionMask, _inputReader, new PlaneListener(Vector3.zero, Vector3.forward)),
        ViewType.View3D => new RaycastListener3D<ISelectable>(sceneCamera, selectionMask, _inputReader),
        _ => _listener
    };
}
```

What the `IListener<T>` does is that it returns a generic object (`ISelectable` in this case) by doing a Raycast in to the scene and try to get the Component from the resulting collider. There is also the option to initialize it with a fallback listener if no collider is detected.

This is important because this is how we select the different elements in our game. And if we look at the `ISelectable` interface we can see that all it does is allow us to return a State.

```
public interface ISelectable
{
    State Select();
    void Deselect();
}
```

If the State rings a bell, it is because we looked at it very briefly when we initialized the Input-Module for our capsule unit. And the State is the base class of all the game loop related states.

So if we now look at the `GameController` object we can see that progressing through the game is handled in the `Progress` method

```
private void Progress()
{
    _currentState ??= DefaultState;
    _currentState = _currentState.Run();
}
```

How does it run?

Lets look at the DefaultState

```
// todo: implement a return state to next  
2 usages 1 Calle  
private State DefaultState => new IdleState(_inputReader, _selectionListener, onDefaultState);
```

This State is simply waiting for input from the IInputReader, and if it is triggered it will try to detect an object that implements ISelectable, and if it finds one, it will return a new State using the Select method on the ISelectable.

We can implement the ISelectable on any object that we want to be able to alter the course of the game loop. A few examples of these are GridEntity (our 'units' base), Cell and GridButton.

So a simple progression of the game might look like:

- Start Game
- Objects get initialized
- GameController listens for input to get a new state
- Player clicks a unit with the GridEntity component
- GridEntity returns a ConfirmMovementState through the Get method
- GameController runs the ConfirmMovementState until it is completed
- If no next state is specified GameController will run DefaultState

